

Compte Rendu

TP Chapitre 2

*Pipelines agiles, versionnement et orchestration
avec **dlt**, **dbt** et **Dagster***

Module : **MLOps & DataOps**

Enseignant : **Pr. Mohammed AIT DAOUD**

Étudiant : **Mouad RADOUANI**

Filière : **IA**

Année universitaire : **2025 – 2026**

Table des matières

Introduction	2
Schéma du pipeline	2
Préparation de l'environnement	2
Initialisation Git	3
1 Ingestion des données	3
2 Validation minimale	4
3 Transformation avec dbt	5
4 Tests dbt	6
5 Orchestration avec Dagster	7
6 CI simple avec GitHub Actions	9
Réponses aux questions	10
Conclusion	11

Introduction

Ce compte rendu présente la réalisation du TP Chapitre 2 du module MLOps & DataOps. L'objectif était de transformer un traitement de données élémentaire en un **pipeline structuré et reproductible**, en mobilisant quatre outils complémentaires :

- **dlt** — ingestion et chargement des données CSV dans DuckDB;
- **dbt** — transformation et tests de qualité des données;
- **Dagster** — orchestration de l'ensemble des étapes;
- **Git** — versionnement du projet.

Le pipeline final suit le flux : CSV → dlt → DuckDB → dbt → Dagster.

Schéma du pipeline

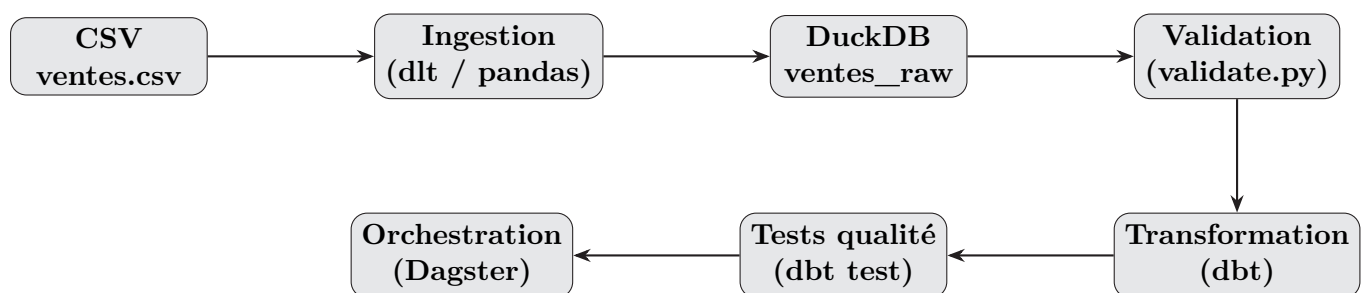


FIGURE 1 – Flux complet du pipeline de données

Préparation de l'environnement

Création du dossier et de l'environnement virtuel

La première étape consiste à créer le répertoire de projet et à initialiser un environnement virtuel Python afin de garantir la **reproductibilité** des dépendances.

```
mkdir tp_chapitre2_pipeline
cd tp_chapitre2_pipeline
python3 -m venv .venv
source .venv/bin/activate
```

```
PS C:\Users\mouad\tp_chapitre2_pipeline> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline> |
```

FIGURE 2 – Activation du venv

Installation des dépendances

```
python -m pip install --upgrade pip
pip install dlt dagster dagster-webserver dbt-core dbt-duckdb pandas duckdb
pyyaml
pip freeze > requirements.txt
```

```
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline> cat requirements.txt
agate==1.9.1
alembic==1.18.4
annotated-types==0.7.0
antlr4-python3-runtime==4.13.2
anyio==4.13.0
attrs==26.1.0
babel==2.18.0
backoff==2.2.1
certifi==2026.5.20
charset-normalizer==3.4.7
click==8.4.1
colorama==0.4.6
coloredlogs==14.0
daff==1.4.2
dagster==1.13.7
dagster-graphql==1.13.7
dagster-pipes==1.13.7
dagster-shared==1.13.7
dagster-webserver==1.13.7
dbt-adapters==1.24.2
dbt-common==1.38.0
dbt-core==1.11.11
dbt-duckdb==1.10.1
```

FIGURE 3 – Installation des packages et génération de requirements.txt

Initialisation Git

```
git init
git config --global init.defaultBranch main
# Cratation du .gitignore puis :
git add .
git commit -m "Initialisation du projet pipeline"
```

Étape 1 — Ingestion des données

Le fichier `pipeline/ingest.py` charge le CSV dans DuckDB via pandas, en créant la table `ventes_raw`.

```
import pandas as pd
import duckdb
from pathlib import Path

csv_path = Path("data/ventes.csv")
```

```
db_path = "ventes.duckdb"

df = pd.read_csv(csv_path)
con = duckdb.connect(db_path)
con.execute("CREATE OR REPLACE TABLE ventes_raw AS SELECT * FROM df")
con.close()
print("Ingestion terminée : ventes_raw créée dans DuckDB")
```

Exécution :

```
python pipeline/ingest.py
```

```
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline> python pipeline/ingest.py
Ingestion terminée : ventes_raw créée dans DuckDB
```

FIGURE 4 – Exécution de ingest.py

Commit :

```
git add pipeline/ingest.py
git commit -m "Ajout de l'étape d'ingestion"
```

Étape 2 — Validation minimale

Le fichier `pipeline/validate.py` vérifie la présence de toutes les colonnes requises et l'absence de valeurs nulles dans les champs critiques.

```
import duckdb

db_path = "ventes.duckdb"
required_columns = {"date", "produit", "categorie",
                    "quantite", "prix_unitaire", "ville"}

con = duckdb.connect(db_path)
columns = con.execute("DESCRIBE ventes_raw").fetchall()
existing_columns = {col[0] for col in columns}

missing = required_columns - existing_columns
if missing:
    raise ValueError(f"Colonnes manquantes : {missing}")

null_count = con.execute("""
    SELECT COUNT(*) FROM ventes_raw
    WHERE produit IS NULL
       OR quantite IS NULL
       OR prix_unitaire IS NULL
""").fetchone()[0]
con.close()

if null_count > 0:
    raise ValueError(f"Données invalides : {null_count} lignes incomplètes")
```

```
else:
    print("Validation reussie : schema et qualite minimale OK")
```

Exécution :

```
python pipeline/validate.py
```

```
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline> python pipeline/validate.py
Validation réussie : schéma et qualité minimale OK
```

FIGURE 5 – Exécution de validate.py

```
git add pipeline/validate.py
git commit -m "Ajout de l'étape de validation minimale"
```

Étape 3 — Transformation avec dbt

Modèle ventes_clean.sql

Ce modèle filtre les lignes valides et calcule le chiffre d'affaires par ligne.

```
select
    date,
    produit,
    categorie,
    quantite,
    prix_unitaire,
    ville,
    quantite * prix_unitaire as chiffre_affaires
from ventes_raw
where quantite > 0 and prix_unitaire > 0
```

Modèle ventes_resume.sql

Ce modèle agrège par catégorie.

```
select
    categorie,
    sum(quantite) as total_quantite,
    sum(chiffre_affaires) as total_ca
from {{ ref('ventes_clean') }}
group by categorie
```

Exécution :

```
cd dbt_pipeline
dbt debug --profiles-dir .
dbt run --profiles-dir .
cd ..
```

```
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline> cd dbt_pipeline
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline\dbt_pipeline> dbt debug --profiles-dir .
00:14:03 Running with dbt=1.11.11
00:14:03 dbt version: 1.11.11
00:14:03 python version: 3.11.9
00:14:03 python path: C:\Users\mouad\tp_chapitre2_pipeline\.venv\Scripts\python.exe
00:14:03 os info: Windows-10-10.0.26200-SP0
00:14:03 Using profiles dir at .
00:14:03 Using profiles.yml file at .\profiles.yml
00:14:03 Using dbt_project.yml file at C:\Users\mouad\tp_chapitre2_pipeline\dbt_pipeline\dbt_project.yml
00:14:03 adapter type: duckdb
00:14:03 adapter version: 1.10.1
00:14:03 Configuration:
00:14:03   profiles.yml file [OK found and valid]
00:14:03   dbt_project.yml file [OK found and valid]
00:14:03 Required dependencies:
00:14:04   - git [OK found]

00:14:04 Connection:
00:14:04   database: ventes
00:14:04   schema: main
00:14:04   path: ../ventes.duckdb
00:14:04   config_options: None
00:14:04   extensions: None
00:14:04   settings: {}
00:14:04   external_root: .
00:14:04   use_credential_provider: None
00:14:04   attach: None
00:14:04   filesystems: None
00:14:04   remote: None
00:14:04   plugins: None
00:14:04   disable_transactions: False
00:14:04 Registered adapter: duckdb=1.10.1
00:14:08 Connection test: [OK connection ok]

00:14:08 All checks passed!
```

FIGURE 6 – dbt debug

```
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline\dbt_pipeline> dbt run --profiles-dir .
00:16:02 Running with dbt=1.11.11
00:16:02 Registered adapter: duckdb=1.10.1
00:16:05 Found 2 models, 3 data tests, 485 macros
00:16:05 Concurrency: 1 threads (target='dev')
00:16:05
00:16:05 1 of 2 START sql table model main.ventes_clean ..... [RUN]
00:16:05 1 of 2 OK created sql table model main.ventes_clean ..... [OK in 0.14s]
00:16:05 2 of 2 START sql table model main.ventes_resume ..... [RUN]
00:16:05 2 of 2 OK created sql table model main.ventes_resume ..... [OK in 0.07s]
00:16:05
00:16:05 Finished running 2 table models in 0 hours 0 minutes and 0.45 seconds (0.45s).
00:16:05 Completed successfully
00:16:05
00:16:05 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
```

FIGURE 7 – dbt run

```
git add dbt_pipeline
git commit -m "Ajout des modeles dbt"
```

Étape 4 — Tests dbt

Le fichier `schema.yml` définit des tests `not_null` sur les colonnes critiques des deux modèles.

```
version: 2
models:
  - name: ventes_clean
```

```
columns:
  - name: produit
    tests: [not_null]
  - name: quantite
    tests: [not_null]
- name: ventes_resume
  columns:
    - name: categorie
      tests: [not_null]
```

Exécution :

```
cd dbt_pipeline
dbt test --profiles-dir .
cd ..
```

```
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline\dbt_pipeline> dbt test --profiles-dir .
00:16:49 Running with dbt=1.11.11
00:16:49 Registered adapter: duckdb=1.10.1
00:16:50 Found 2 models, 3 data tests, 485 macros
00:16:50
00:16:50 Concurrency: 1 threads (target='dev')
00:16:50
00:16:50 1 of 3 START test not_null_ventes_clean_produit ..... [RUN]
00:16:50 1 of 3 PASS not_null_ventes_clean_produit ..... [PASS in 0.06s]
00:16:50 2 of 3 START test not_null_ventes_clean_quantite ..... [RUN]
00:16:50 2 of 3 PASS not_null_ventes_clean_quantite ..... [PASS in 0.03s]
00:16:50 3 of 3 START test not_null_ventes_resume_categorie ..... [RUN]
00:16:50 3 of 3 PASS not_null_ventes_resume_categorie ..... [PASS in 0.02s]
00:16:50
00:16:50 Finished running 3 data tests in 0 hours 0 minutes and 0.27 seconds (0.27s).
00:16:50
00:16:50 Completed successfully
00:16:50
00:16:50 Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=3
```

FIGURE 8 – dbt test

```
git add dbt_pipeline/models/schema.yml
git commit -m "Ajout des tests simples dbt"
```

Étape 5 — Orchestration avec Dagster

Le fichier `pipeline/orchestrate.py` définit quatre *ops* enchaînés dans un *job* Dagster.

```
from dagster import job, op
import os

@op
def ingest():
    os.system("python pipeline/ingest.py")

@op
def validate():
    os.system("python pipeline/validate.py")

@op
def transform():
```



```

os.system("cd dbt_pipeline && dbt run --profiles-dir .")

@op
def test_data():
    os.system("cd dbt_pipeline && dbt test --profiles-dir .")

@job
def ventes_pipeline():
    test_data(transform(validate(ingest()))))

```

Lancement de l'interface Dagster :

```
dagster dev -f pipeline/orchestrate.py
```

```

(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline> dagster dev -f pipeline/orchestrate.py
C:\Users\mouad\tp_chapitre2_pipeline\.venv\Lib\site-packages\click\core.py:877: SupersessionWarning: Function 'dev_command' is superseded and its usage is discouraged. Use 'dag dev' instead.
  return callback(*args, **kwargs)
2026-06-02 01:18:00 +0100 - dagster - INFO - Using temporary directory C:\Users\mouad\tp_chapitre2_pipeline\.tmp_dagster_home_vdty5em5 for storage. This will be removed when dagster dev exits.
2026-06-02 01:18:00 +0100 - dagster - INFO - To persist information across sessions, set the environment variable DAGSTER_HOME to a directory to use.
2026-06-02 01:18:03 +0100 - dagster - INFO - Launching Dagster services...
2026-06-02 01:18:09 +0100 - dagster.daemon - INFO - Instance is configured with the following daemons: ['AssetDaemon', 'BackfillDaemon', 'FreshnessDaemon', 'QueuedRunCoordinatorDaemon', 'SchedulerDaemon', 'SensorDaemon']
2026-06-02 01:18:11 +0100 - dagster - WARNING - C:\Users\mouad\tp_chapitre2_pipeline\.venv\Lib\site-packages\dagster_core\execution\compute_logs.py:53: UserWarning: WARNING: Compute log capture is disabled for the current environment. Set the environment variable 'PYTHONLEGACYWINDOWSTDIO' to enable.
  warnings.warn(WIN_PY36_COMPUTE_LOG_DISABLED_MSG)
2026-06-02 01:18:11 +0100 - dagster-webserver - INFO - Serving dagster-webserver on http://127.0.0.1:3000 in process 4056

```

FIGURE 9 – Lancement de Dagster

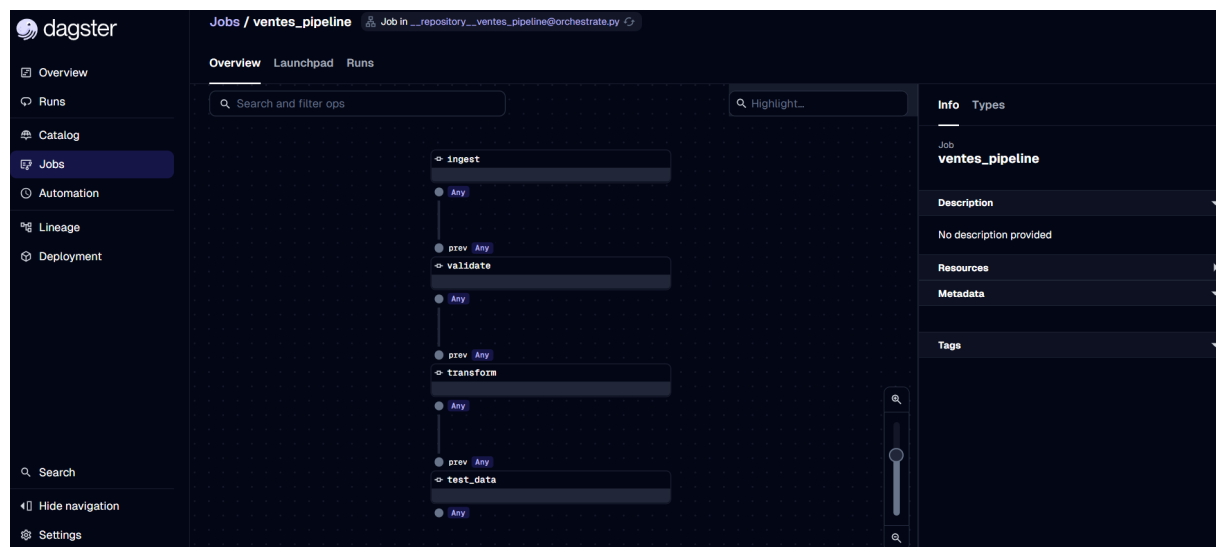


FIGURE 10 – Interface web Dagster (graphe du pipeline)

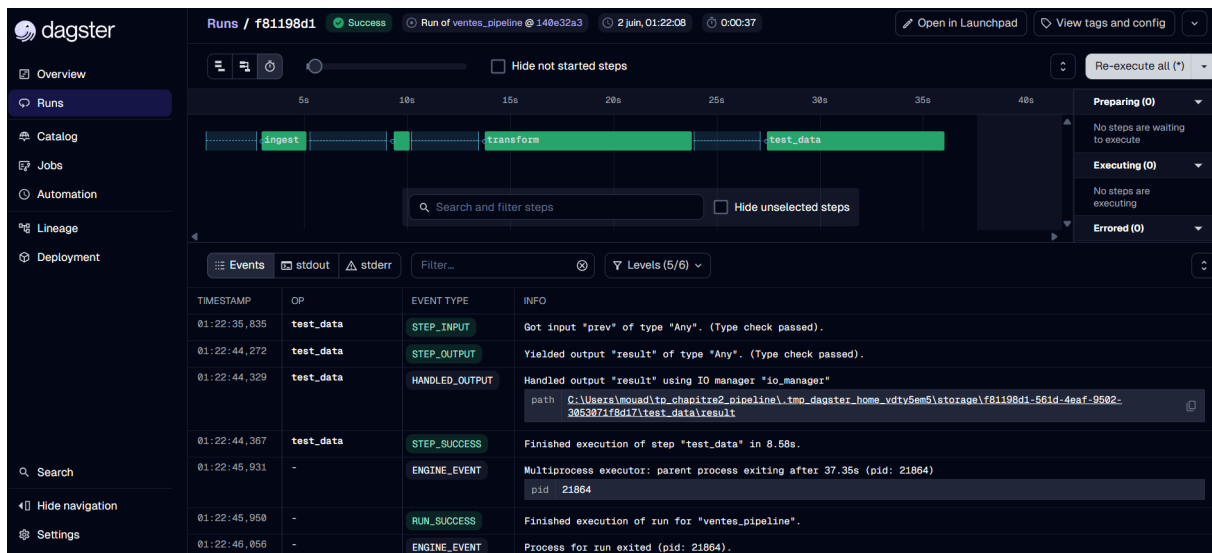


FIGURE 11 – Exécution réussie du pipeline dans Dagster

```
git add pipeline/orchestrate.py
git commit -m "Ajout de l'orchestration Dagster"
```

Étape 6 — CI simple avec GitHub Actions

Le fichier `.github/workflows/ci.yml` déclenche automatiquement une vérification syntaxique à chaque *push* ou *pull request* sur la branche `main`.

```
name: CI Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  validate-project:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Syntax check
        run: |
          python -m py_compile pipeline/ingest.py \
            pipeline/validate.py \
            pipeline/orchestrate.py
```

```
(.venv) PS C:\Users\mouad\tp_chapitre2_pipeline> git log --oneline
d8f5d9e (HEAD -> master, origin/master) Ajout d'une CI simple
f79db1a Ajout de l'orchestration Dagster
322cc7e Ajout des tests simples dbt
6ee058d Ajout des modèles dbt
3d2f2a2 Ajout de l'étape de validation minimale
48c5d73 Ajout de l'étape d'ingestion
6a09c08 Initialisation du projet pipeline
```

FIGURE 12 – Historique Git complet

```
git add .github/workflows/ci.yml
git commit -m "Ajout d'une CI simple"
```

Réponses aux questions

Q1. En quoi ce pipeline est-il plus structuré qu'un notebook unique ?

Un notebook unique mélange dans le même fichier la logique de chargement, de transformation et de visualisation, ce qui le rend difficile à tester, à maintenir et à réexécuter de façon fiable. Le pipeline structuré sépare chaque préoccupation en un fichier dédié (`ingest.py`, `validate.py`, modèles dbt, `orchestrate.py`), ce qui facilite la détection d'erreurs, la réutilisation et la collaboration.

Q2. Quel est le rôle de Git dans ce projet ?

Git assure le **versionnement** du code : il trace l'historique de chaque modification, permet de revenir à un état antérieur en cas de régression, et facilite le travail en équipe via les branches et les pull requests. Chaque étape du TP est matérialisée par un commit distinct, constituant une trace d'audit du développement.

Q3. Quel est le rôle de `requirements.txt` ?

Il fige les versions exactes de toutes les dépendances Python installées, ce qui garantit que n'importe quel autre développeur — ou un serveur CI — peut reproduire exactement le même environnement via `pip install -r requirements.txt`. C'est la base de la **reproductibilité**.

Q4. Pourquoi ajouter une étape `validate.py` ?

Sans validation, une donnée corrompue (colonne manquante, valeur nulle) se propagera silencieusement jusqu'aux modèles dbt et produira des résultats erronés sans qu'une erreur explicite soit levée. `validate.py` joue le rôle d'**interruption rapide** (*fail fast*) : il bloque le pipeline dès la source si les données ne respectent pas le contrat de qualité attendu.

Q5. Quel est le rôle de `dbt test` ?

`dbt test` applique des assertions déclaratives sur les données transformées (ici `not_null`) directement dans la base DuckDB. Il complète `validate.py` en contrôlant la qualité *après* transformation, et produit un rapport lisible qui s'intègre naturellement à un pipeline CI/CD.

Q6. Qu'apporte Dagster par rapport à une exécution manuelle ?

Une exécution manuelle (`python ingest.py && python validate.py && ...`) est fragile et non traçable. Dagster offre : une **interface graphique** pour visualiser le graphe des dépendances, des **logs centralisés** par étape, la **gestion des erreurs** avec possibilité de relancer uniquement les étapes échouées, et une base solide pour planifier les exécutions (*scheduling*).

Q7. Qu'apporte la CI ?

La CI exécute automatiquement les vérifications syntaxiques (et potentiellement les tests) à chaque modification du code poussée sur le dépôt. Elle garantit qu'aucune régression n'est introduite sans être détectée, et rend le projet **toujours dans un état déployable**.

Q8. Que manque-t-il encore pour un pipeline pleinement industrialisé ?

- Un **entrepôt de données** cloud (Snowflake, BigQuery) à la place de DuckDB local ;
- Des **alertes** (email, Slack) en cas d'échec du pipeline ;
- La **gestion des secrets** (credentials, tokens) via un coffre-fort ;
- Un **catalogue de données** (dbt docs, DataHub) pour la documentation ;
- Des **tests de régression** et un monitoring des dérives de données ;
- Un **environnement de staging** distinct de la production ;
- L'**automatisation du déploiement** (CD) en complément de la CI.

Conclusion

Ce TP a permis de mettre en œuvre un pipeline de données local complet, structuré en étapes distinctes et orchestré de bout en bout. L'utilisation conjointe de **dlt/pandas**, **dbt**, **Dagster** et **Git** illustre les bonnes pratiques MLOps/DataOps : séparation des responsabilités, reproductibilité, traçabilité et automatisation. Ce socle constitue une base solide pour évoluer vers des pipelines industriels à plus grande échelle.